EcAuTAL 2026 · LISN · TP de 40 minutes

TP « vibe coding » : un outil de TAL pour le grec ancien

avec pi-aristote, l'agent de code branché sur les modèles d'Aristote

ΣΠΕΥΔΕ ΒΡΑΔΕΩΣ · σπεῦδε βραδέως, hâte-toi lentement

Ici, c'est l'agent qui écrit le code : vous demandez à pi, branché sur les modèles d'Aristote, de construire un petit outil de TAL pour le grec ancien, palier par palier, vous lui faites lancer les tests et vous relisez ce qu'il a écrit. Il y a cinq paliers, et chacun a un test qui affiche « réussi » quand le palier est fini. Le corpus réunit neuf textes grecs (41 000 mots) : Homère (*Iliade* 1 et 18, *Odyssée* 11), Platon (*Euthyphron*, *Apologie*, *Criton*), Xénophon (*Apologie*) et Lysias (discours 1 et 12), dans les éditions de The Greek Library (Perseus, CC BY-SA 4.0).

Avant de commencer

1. Récupérer une clé Aristote : connectez-vous sur `https://aristote.education/llm-access/` avec votre compte universitaire, puis copiez la clé (elle commence par `sk-`). Elle est personnelle : ne la montrez à personne, pas même à l'agent.
2. Installer pi-aristote, en une commande dans un terminal. Sous macOS ou Linux :

   ```
   curl -fsSL https://thegreeklibrary.org/pi-aristote/install.sh | sh
   ```

   Sous Windows, dans PowerShell :

   ```
   wsl bash -lc "curl -fsSL https://thegreeklibrary.org/pi-aristote/install.sh | sh"
   ```

   La première fois sous Windows, il faut d'abord taper `wsl --install` dans un PowerShell ouvert en administrateur, redémarrer, puis choisir un nom d'utilisateur Linux. Ensuite, **tout le TP se fait dans WSL** (tapez `wsl`), pas dans PowerShell. L'installateur demande la clé et la range dans `~/.pi/agent` (détails sur `thegreeklibrary.org/pi-aristote`).
3. Vérifier que vous avez `python3` (3.8 ou plus) et `git`. Il n'y a rien d'autre à installer, le TP n'utilise que la bibliothèque standard. Sous macOS, la première commande `python3` propose d'installer les outils de ligne de commande : acceptez (ça prend quelques minutes).

Une fois tout installé, lancez `pi` et tapez `/aristote` : la liste des modèles doit s'afficher (sinon, `/aristote key sk-…` avec votre clé).

Démarrer (3 minutes)

Taper ces commandes dans le terminal où pi fonctionne (sous Windows, dans WSL et dans votre dossier ~, pas dans `/mnt/c`). Le kit est aussi en lien sur la page du cours, `thegreeklibrary.org/pi-aristote/ecautal`.

```
curl -fsSLO https://thegreeklibrary.org/pi-aristote/ecautal/tp-pi-aristote.zip
python3 -m zipfile -e tp-pi-aristote.zip .
cd tp-pi-aristote
git init -q && git add -A
git -c user.name=moi -c user.email=moi@example.org commit -qm "kit de départ"
python3 tests/tout.py
pi
```

`tests/tout.py` affiche « à faire » pour les cinq paliers, et grâce au commit de départ, `git diff` vous montrera ensuite tout ce que l'agent a changé. Le kit contient `corpus/` (les textes et les vedettes du LSJ), `tests/` (un test par palier) et `AGENTS.md`, qui décrit ce qui est attendu à chaque palier : pi le lit en démarrant, lisez-le aussi. Ne modifiez ni `corpus/` ni `tests/`.

Les paliers

Allez aussi loin que possible, un palier après l'autre (le palier 3 est déjà un bon résultat, et les durées sont indicatives). Dans chaque palier, l'encadré orange est le *prompt* à taper (ou à coller) dans pi, et l'encadré gris une commande à lancer dans le terminal.

α Palier 1 : Fréquences — 10 min

Normaliser le grec (NFC et NFD, accent grave, oxia et tonos, apostrophes `' ’ ʼ`, sigma final), puis compter les mots du corpus.

> Lis AGENTS.md. Fais le palier 1 : écris grec.py et frequences.py, puis lance python3 tests/palier1.py et corrige jusqu'à ce qu'il soit réussi. Montre-moi les 10 mots les plus fréquents de l'Apologie de Platon.

Vérifier avec `python3 tests/palier1.py`, puis `python3 frequences.py --texte platon-apologie`.

Indice : lisez les `[ÉCHEC]`*, ils montrent les caractères en cause.*

Regarder ensuite pourquoi ὦ est si fréquent dans l'*Apologie*, et pourquoi δ' passe devant δέ dans l'*Odyssée* (`--texte homere-odyssee-11`).

Pour aller plus loin : option `--sans-accents` (pourquoi η devient-il le 4e mot ?), tri alphabétique grec, hapax, loi de Zipf.

β Palier 2 : Concordancier — 8 min

Afficher chaque occurrence d'un mot dans son contexte, avec sa référence (`Pl. Ap. 28d`).

> Palier 2 : écris concordance.py comme le décrit AGENTS.md, en réutilisant grec.py. Lance python3 tests/palier2.py. Puis montre-moi la concordance de ψυχ* : que remarques-tu entre Homère et Platon ?

Vérifier avec `python3 tests/palier2.py`, puis essayer `python3 concordance.py τεθναίην` (Socrate cite Achille) et `python3 concordance.py Μίνως`. Regarder ce que devient ψυχή entre la Nékyia et l'*Apologie*, et pourquoi *Odyssée* 11.568 n'apparaît pas pour Μίνως.

Indice : essayez `python3 concordance.py 'Μίνω*'` *(avec les guillemets, à cause de l'étoile).*

Pour aller plus loin : recherche sans accents (`ομηρ*` retrouve Ὁμήρῳ), voisins les plus fréquents de ψυχή.

γ Palier 3 : Page web — 8 min

Faire la même recherche dans le navigateur, sans serveur (la page s'ouvre d'un double-clic sur `site/index.html`).

> Palier 3 : construis la page web (construire_site.py, site/index.html, site/recherche.js) décrite dans AGENTS.md. Lance python3 tests/palier3.py, puis ouvre site/index.html avec ton navigateur et montre-la-moi.

Vérifier avec `python3 tests/palier3.py`, puis ouvrir la page vous-même : `open site/index.html` (macOS), `xdg-open site/index.html` (Linux), `explorer.exe "$(wslpath -w site/index.html)"` (WSL). Chercher τεθναίην puis ψυχ*, et comparer avec ce que donnait le palier 2.

Pour aller plus loin : un lien de chaque référence vers The Greek Library (champ `tgl_url` de `corpus/metadonnees.json`), un filtre par auteur, la saisie en bêtacode (`yuxh/` donne ψυχή).

δ Palier 4 : Lemmatisation avec Aristote — 10 min

Demander les lemmes à un LLM, les vérifier dans le dictionnaire LSJ, relancer le modèle quand le lemme n'y est pas, et garder les réponses brutes.

> Palier 4 : écris lemmatiser.py comme le décrit AGENTS.md (API Aristote en bibliothèque standard, clé jamais affichée, cache, vérification dans le LSJ et relance). Lance python3 lemmatiser.py tests/donnees/palier4.txt > sorties/palier4.tsv, puis python3 tests/palier4.py.

Vérifier avec `python3 tests/palier4.py` (il n'appelle pas l'API, vous pouvez le relancer à volonté) : il faut au moins 70 % de lemmes justes sur 111 mots. L'appel au modèle, lui, prend de quelques secondes à une minute par phrase. Regarder quels mots sont faux, et s'ils sont plutôt chez Homère ou chez Lysias.

Pour aller plus loin : comparer les modèles d'Aristote (par exemple `--modele gemma-4-31b`), lemmatiser l'*Apologie* entière et chercher par lemme : Μίνως (Ap. 41a) et Μίνωα (Od. 11.568).

ε Palier 5 : Serveur MCP

bonus

Faire en sorte que pi interroge lui-même le corpus, par le protocole MCP vu en cours.

> Palier 5 : écris serveur_mcp.py comme le décrit AGENTS.md et lance python3 tests/palier5.py.

Puis quitter pi (`/exit`) et brancher le serveur :

```
pi mcp add corpus --exposure direct -- python3 "$PWD/serveur_mcp.py"
pi mcp list
pi
```

Puis, dans pi :

> Avec l'outil corpus, compare les emplois de ψυχή chez Homère et chez Platon.

Après le TP, débranchez-le avec `pi mcp remove corpus`.

Pour aller plus loin : essayer les sous-agents d'Ultracode avec « ultracode : compare le vocabulaire des deux *Apologies* (Platon, Xénophon) ; un disciple par texte, Socrate vérifie les chiffres avec frequences.py ».

Bien piloter l'agent

- Un palier par demande : « Fais tout le TP » tourne longtemps et se perd.
- Faites lancer le test et lisez sa sortie : les `[ÉCHEC]` disent ce qui manque.
- Relisez le code et `git diff` : vous êtes responsable de ce que l'agent écrit (`git checkout -- fichier` annule une modification).
- Tapez `/clear` entre deux paliers : le contexte est plus court et les réponses plus rapides.
- Échap interrompt l'agent, et `/model` change de modèle si celui-ci est trop lent.
- Avec `/socrate lance tous les tests et relis concordance.py`, un disciple relit sans rien modifier.
- Ne laissez jamais l'agent afficher votre clé, ni modifier `corpus/` ou `tests/`.

En cas de problème

- `python3` introuvable (macOS) : acceptez l'installation des outils de ligne de commande.
- Pas de `unzip` : `python3 -m zipfile -e tp-pi-aristote.zip .` marche partout.
- Zip téléchargé avec le navigateur, sous WSL : il est dans `/mnt/c/Users/<nom>/Downloads/` ; copiez-le dans ~ avant de le décompresser.
- Erreur 401 : la clé a expiré ou a été révoquée ; générez-en une nouvelle sur `aristote.education/llm-access/`, puis `/aristote key sk-…`
- Le modèle ne répond plus : patientez une minute (tout le monde l'interroge en même temps), puis `/model`.
- « le corpus n'a pas été modifié » en échec : `git checkout -- corpus/`.
- Grec mal affiché dans le terminal (des carrés) : prenez une police qui couvre le grec polytonique (DejaVu Sans Mono) ; les fichiers, eux, sont justes.

À montrer au bilan (2 h 25)

rien à rendre par écrit

On fait le point à main levée, palier par palier. Montrez :

1. la sortie de `python3 tests/tout.py` ;
2. un résultat à l'écran, au choix : la concordance de ψυχ* chez Homère et chez Platon, les deux occurrences de τεθναίην, votre page web, ou votre score du palier 4 avec un lemme faux qui vous a surpris ;
3. une erreur de l'agent, et la façon dont vous l'avez repérée (un test, `git diff`, une relecture).

Pour garder votre travail, faites un dernier commit :

```
git add -A && git -c user.name=moi -c user.email=moi@example.org commit -qm "fin du TP"
```

Textes : Perseus Digital Library, CC BY-SA 4.0, via The Greek Library (`thegreeklibrary.org`). Vedettes du LSJ : Perseus (CC BY-SA 4.0) via Eulexis, voir `corpus/LICENCE.md`. La page du palier 3 embarque les textes : si vous la publiez, c'est sous la même licence. Diapositives, fiche et kit : `thegreeklibrary.org/pi-aristote/ecautal`.